```
# ================================
# INSTALL (Run once)
# ================================
# !pip install transformers datasets accelerate -q

# ================================
# IMPORTS
# ================================
import torch, math, sys
from transformers import (
    GPT2LMHeadModel,
    GPT2Tokenizer,
    Trainer,
    TrainingArguments,
    DataCollatorForLanguageModeling,
    set_seed
)
from datasets import Dataset

# Fix stdout issue
sys.stdout = sys.__stdout__

set_seed(42)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


# ================================
# TEXT GENERATION FUNCTION
# ================================
def generate_text(model, tokenizer, prompt, max_length=100):
    model.eval()
    inputs = tokenizer.encode(prompt, return_tensors='pt').to(device)

    with torch.no_grad():
        output = model.generate(
            inputs,
            max_length=max_length,
            temperature=0.8,
            top_k=50,
            top_p=0.95,
            do_sample=True,
            pad_token_id=tokenizer.eos_token_id
        )

    return tokenizer.decode(output[0], skip_special_tokens=True)


# ========================================================
# 🛒 COMPONENT I: PRODUCT REVIEW GENERATOR
# ========================================================

print("\n=== COMPONENT I: PRODUCT REVIEWS ===")

# Load GPT-2
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')
model = GPT2LMHeadModel.from_pretrained('gpt2').to(device)

tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"
model.config.pad_token_id = tokenizer.eos_token_id


# --------------------------------
# BASELINE
# --------------------------------
review_prompts = [
    "This product is",
    "I bought this phone and",
    "The quality of this item"
]

baseline = {}
print("\n=== BASELINE REVIEWS ===")
for p in review_prompts:
    baseline[p] = generate_text(model, tokenizer, p)
    print(f"\nPrompt: {p}\nOutput: {baseline[p]}")


# --------------------------------
# DATASET
```

```
# DATASET
# -------------------------------
corpus = [
    "this phone has an amazing battery life and the camera quality is outstanding for the price.",
    "i bought this laptop for college and it handles all my assignments and coding projects perfectly.",
    "the sound quality of these headphones is incredible with deep bass and clear vocals.",
    "this smartwatch tracks my steps accurately and the heart rate monitor is very reliable.",
    "great wireless earbuds with noise cancellation that blocks out all background sound.",
    "the keyboard feels very comfortable for long typing sessions and the backlight is a nice touch.",
    "this portable charger saved me during travel and it charges my phone three times on a single charge.",
    "the tablet screen is bright and colorful which makes watching movies a great experience.",
    "i love this fitness tracker because it motivates me to reach my daily exercise goals.",
    "this bluetooth speaker is compact but delivers surprisingly loud and clear audio.",
    "the delivery was fast and the product was packed securely with no damage at all.",
    "excellent value for money and the build quality feels premium despite the affordable price.",
    "the customer service team was very helpful when i had questions about the product features.",
    "this camera takes stunning photos in low light and the video recording quality is very smooth.",
    "i have been using this product for three months and it still works perfectly like day one.",
    "the design is sleek and modern and it looks great on my desk next to my other gadgets.",
    "easy to set up right out of the box and the instructions were clear and simple to follow.",
    "highly recommend this product to anyone looking for quality and reliability at a fair price.",
    "the software updates keep adding new features which makes this purchase even more worthwhile.",
    "best purchase i made this year and i would definitely buy from this brand again."
]

dataset = Dataset.from_dict({"text": corpus})

tokenized = dataset.map(
    lambda x: tokenizer(
        x["text"],
        truncation=True,
        max_length=128,
        padding="max_length"
    ),
    batched=True,
    remove_columns=["text"]
)

split = tokenized.train_test_split(test_size=0.15, seed=42)


# -------------------------------
# TRAINING
# -------------------------------
collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

training_args = TrainingArguments(
    output_dir="./gpt2-reviews",
    num_train_epochs=5,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    weight_decay=0.01,
    warmup_steps=50,
    eval_strategy="epoch",   # for your version
    logging_steps=10,
    save_strategy="no",
    fp16=torch.cuda.is_available(),
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=split["train"],
    eval_dataset=split["test"],
    data_collator=collator
)

print("\nTraining reviews model...")
trainer.train()


# -------------------------------
# EVALUATION (FIXED)
# -------------------------------
metrics = trainer.predict(split["test"])
loss = metrics.metrics["test_loss"]
print(f"\nPerplexity: {math.exp(loss):.2f}")


print("\n=== FINE-TUNED REVIEWS ===")
for p in review_prompts:
    out = generate_text(model, tokenizer, p)
```

```
    print(f"\nPrompt: {p}")
    print(f"Baseline: {baseline[p][:100]}")
    print(f"Fine-Tuned: {out[:100]}")


# ========================================================
# 🍝 COMPONENT II: RECIPE GENERATOR
# ========================================================

print("\n=== COMPONENT II: RECIPES ===")

# Load fresh GPT-2
tokenizer2 = GPT2Tokenizer.from_pretrained('gpt2')
model2 = GPT2LMHeadModel.from_pretrained('gpt2').to(device)

tokenizer2.pad_token = tokenizer2.eos_token
tokenizer2.padding_side = "right"
model2.config.pad_token_id = tokenizer2.eos_token_id


# -------------------------------
# BASELINE
# -------------------------------
recipe_prompts = [
    "To make butter chicken",
    "For pasta carbonara",
    "To prepare a chocolate cake"
]

baseline2 = {}
print("\n=== BASELINE RECIPES ===")
for p in recipe_prompts:
    baseline2[p] = generate_text(model2, tokenizer2, p)
    print(f"\nPrompt: {p}\nOutput: {baseline2[p]}")


# -------------------------------
# DATASET
# -------------------------------
recipes = [
    "to make butter chicken start by marinating chicken pieces in yogurt with turmeric chili powder and garam masala for one
    "heat butter in a pan and fry onions until golden brown then add ginger garlic paste and cook for two minutes.",
    "add tomato puree and cook on low heat for ten minutes until the oil separates from the masala.",
    "add the marinated chicken and cook on medium heat for fifteen minutes until fully cooked.",
    "finish with fresh cream and kasuri methi and serve hot with naan or steamed rice.",
    "for pasta carbonara boil spaghetti in salted water until al dente and reserve half cup of pasta water.",
    "fry diced pancetta in olive oil until crispy and set aside.",
    "whisk together egg yolks parmesan cheese and black pepper in a bowl.",
    "toss the hot pasta with pancetta and remove from heat then quickly stir in the egg mixture.",
    "the residual heat will cook the eggs into a creamy sauce and serve immediately with extra parmesan.",
    "to prepare vegetable stir fry heat sesame oil in a wok on high heat.",
    "add sliced bell peppers broccoli florets and snap peas and toss for three minutes.",
    "pour in soy sauce oyster sauce and a pinch of sugar and stir well.",
    "add minced garlic and ginger and cook for one more minute until fragrant.",
    "serve the stir fry over steamed jasmine rice and garnish with sesame seeds.",
    "for chocolate chip cookies cream together butter and sugar until light and fluffy.",
    "beat in eggs one at a time then add vanilla extract and mix well.",
    "fold in flour baking soda and salt then gently stir in chocolate chips.",
    "scoop dough onto a baking sheet and bake at 180 degrees for twelve minutes until golden.",
    "let cookies cool on the tray for five minutes before transferring to a wire rack."
]

dataset2 = Dataset.from_dict({"text": recipes})

tokenized2 = dataset2.map(
    lambda x: tokenizer2(
        x["text"],
        truncation=True,
        max_length=128,
        padding="max_length"
    ),
    batched=True,
    remove_columns=["text"]
)

split2 = tokenized2.train_test_split(test_size=0.15, seed=42)


# -------------------------------
# TRAINING
# -------------------------------
collator2 = DataCollatorForLanguageModeling(tokenizer=tokenizer2, mlm=False)
```

```
args2 = TrainingArguments(
    output_dir="./gpt2-recipes",
    num_train_epochs=5,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    weight_decay=0.01,
    warmup_steps=50,
    eval_strategy="epoch",
    logging_steps=10,
    save_strategy="no",
    fp16=torch.cuda.is_available(),
    report_to="none"
)

trainer2 = Trainer(
    model=model2,
    args=args2,
    train_dataset=split2["train"],
    eval_dataset=split2["test"],
    data_collator=collator2
)

print("\nTraining recipes model...")
trainer2.train()


# -------------------------------
# EVALUATION (FIXED)
# -------------------------------
metrics2 = trainer2.predict(split2["test"])
loss2 = metrics2.metrics["test_loss"]
print(f"\nPerplexity: {math.exp(loss2):.2f}")


print("\n=== FINE-TUNED RECIPES ===")
for p in recipe_prompts:
    out = generate_text(model2, tokenizer2, p)
    print(f"\nPrompt: {p}")
    print(f"Baseline: {baseline2[p][:100]}")
    print(f"Fine-Tuned: {out[:100]}")


print("\n✅ LAB COMPLETED SUCCESSFULLY (NO ERRORS)")
```

Loading weights: 100% 148/148 [00:00<00:00, 5201.06it/s]

Map: 100% 20/20 [00:00<00:00, 681.87 examples/s]

[45/45 00:03, Epoch 5/5]

Epoch	Training Loss	Validation Loss
1	No log	3.297611
2	3.965284	3.088262
3	3.587046	2.884873
4	2.839673	2.778178
5	2.089447	2.800223

Loading weights: 100% 148/148 [00:00<00:00, 4545.14it/s]

Map: 100% 20/20 [00:00<00:00, 824.26 examples/s]

[45/45 00:03, Epoch 5/5]

Epoch	Training Loss	Validation Loss
1	No log	3.323145
2	3.964751	3.043005
3	3.531795	3.015297
4	2.843761	2.921232
5	2.217608	2.916104